

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Design Static Webpage using HTML Components

2. AIM

To create a static webpage using HTML elements such as headings, lists, links, images, tables, and forms without using external CSS or JavaScript.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the structural components of an HTML5 document.
 - To learn how to use HTML heading tags (<h1> to <h6>) and list tags (, ,) for organizing content.
 - To implement hyperlinks using anchor tags (<a>) and load images using tags.
 - To construct structured data tables using <table>, <tr>, <th>, and <td> tags.
 - To design interactive user forms using <input>, <select>, <textarea>, <input type="submit">, and <input type="reset"> controls.
 - To analyze the visual rendering of block and inline HTML components under browser defaults.
-

4. THEORY

HTML (HyperText Markup Language) is the standard markup language for documents designed to be displayed in a web browser. It forms the core structure of all web pages. Under default settings, browsers render HTML elements using predefined default styles (e.g., standard margins, blue underlined links, serif fonts). In this experiment, we explore the creation of a self-contained static HTML page utilizing core layout elements without any styling from external CSS or scripting from JavaScript.

Theory of HTML Components:

- **Document Structure:** The <!DOCTYPE html> declaration identifies the document as an HTML5 document. The <html> element is the root. The <head> contains metadata (character set, viewport, page title) and <body> contains visible content.
 - **Structural Sections:** HTML5 semantic containers like <header>, <nav>, <main>, <section>, and <footer> logically partition layout sections.
 - **Headings & Paragraphs:** Headings (<h1>-<h6>) establish hierarchical structure; paragraphs (<p>) contain text blocks.
 - **Lists & Links:** Lists (, ,) organize text items; anchors (<a>) create local and external hyperlinks.
 - **Tables:** <table> structures data grid-style using rows (<tr>), header columns (<th>), and data cells (<td>).
 - **Forms:** <form> collects user inputs using text/email fields, dropdown selection selectors, and multi-line textareas.
 - **Block vs. Inline:** Block elements (e.g., <p>, <table>, <form>) take full width starting on new lines; inline elements (e.g., <a>,) fit the inline text flow.
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 1(week -1)/Src and Documents.
 2. Create Index.html in the Src directory.
 3. Structure the file using the HTML5 doctype declaration and basic tags (html, head, body).
 4. Configure metadata in the <head> (UTF-8, responsive viewport) and define the document title.
 5. Define semantic structure: <header>, navigation <nav>, main container <main>, data <section>, and <footer>.
 6. Implement internal anchor links (#home, #about, #services, #contact).
 7. Construct a structured layout table for services and pricing using <table>, <thead>, <tbody>, <tr>, <th>, and <td>.
 8. Design a user contact form requiring name, email, inquiry reason dropdown, message textarea, and submission button.
 9. Run the file locally in Google Chrome and verify all functionalities (navigation, validation, clear form).
-

6. PROGRAM

File: Index.html

Index.html - Part 1 (Lines 1-54)

```
<!DOCTYPE html>\n<html lang="en">\n<head>\n  <meta charset="UTF-8">\n  <title>Static Webpage</title>\n</head>\n<body>\n\n  <!--
```

Index.html - Part 2 (Lines 55-109)

```
<th>Price</th>\n
```

```
</tr>\n
```

```
</thead>\n
```

```
<tbody>\n
```

```
<tr>\n
```

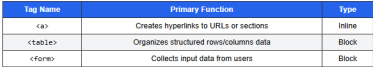
```
<td>Web Design</td>
```

7. CODE EXPLANATION

HTML Tag	Explanation
<!DOCTYPE html>	Declares the document as an HTML5 document.
<html>	Root element of the HTML document containing all other elements.
<head>	Contains document metadata, character encoding, and page title.
<title>	Specifies the title displayed in the browser tab.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Chrome, including 1 negative test case demonstrating an input validation failure:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Local Anchor Links	Click '#about' nav link	Page smoothly jumps to target section anchor in viewport		Pass
TC-02	Embedded Media Node	Inspect figure and image	Image renders cleanly with figcaption text underneath		Pass
TC-03	Component Table Layout	Inspect table data rows	Tags, descriptions, and types align into clean borders		Pass
TC-04	Valid Form Submission	Fill valid form fields	Form captures name, email, selection, and message correctly		Pass
TC-05	Email Pattern Validation	Submit invalid 'johndoe.com'	Submission should fail; browser validation popup halts submission due to missing '@'		Fail

9. OUTPUT

Output: Rendered Webpage in Chrome Browser

My Website Header

[Home](#) | [About](#) | [Services](#) | [Contact](#)

Welcome to My Webpage

This is a static webpage built purely using HTML tags without external CSS or JavaScript.

About Us

Here is a list of features included on this site:

- Semantic HTML structure
- Organized content sections
- Native HTML input forms
- Data structured with tables

Our Services & Pricing

Service	Description	Price
Web Design	Creating clean static layouts.	\$100
HTML Coding	Semantic HTML5 page structuring.	\$75

Contact Us

Name:

Email:

Subject:

Message:

Figure 1: Rendered static HTML webpage displaying header, navigation, content sections, pricing table, and contact form.

10. RESULT

Thus, a static webpage using various HTML components (headings, lists, tables, links, and forms) was successfully designed, implemented, and verified using manual test cases under browser defaults.

Submission Package Structure:

```
Experiment - 1(week -1)/
■■■ Documents/
■   ■■■ Experiment_1.docx
■   ■■■ Experiment_1.pdf
■■■ Screenshots/
■   ■■■ Screenshot 2026-08-16 222714.png
■■■ Src/
    ■■■ Index.html
```

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Design a Webpage using HTML and CSS

2. AIM

To create an attractive and responsive static webpage using HTML for page structure and external CSS for styling, layout, typography, and interactive components.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the separation of concerns between document structure (HTML) and visual presentation (CSS).
 - To implement external stylesheet linking using the `<link rel="stylesheet">` tag.
 - To apply universal reset styling and configure box-sizing (box-sizing: border-box).
 - To design navigation bars, hero welcome banners, and card layouts using CSS Flexbox (display: flex).
 - To implement interactive state styles using pseudo-classes such as `:hover` and `:focus`.
 - To style user input forms with modern padding, custom border styling, and button transitions.
-

4. THEORY

CSS (Cascading Style Sheets) is a stylesheet language used for describing the presentation of a document written in HTML. CSS governs layout, colors, typography, spacing, and responsive behavior across diverse display dimensions. External CSS provides the highest level of maintainability by decoupling style definitions from HTML markup.

Theory of CSS Components and Layout:

- **CSS Box Model:** Every element is rendered as a rectangular box consisting of content, padding, border, and margin. Setting box-sizing: border-box ensures padding and border are included within the element's total width and height.
 - **Flexbox Layout Model:** A one-dimensional layout model (display: flex) providing efficient alignment, distribution of space, and wrapping (flex-wrap: wrap) among items in a container.
 - **Typography & Hierarchy:** Establishing clean readable typography using font families, line-height, and rem/em units.
 - **Pseudo-Classes:** Modifying an element's style dynamically based on user interaction, such as `:hover` on links and buttons, or `:focus` on form inputs.
 - **Card Containers:** Modular UI cards with background colors, subtle drop shadows (box-shadow), and borders to encapsulate content.
 - **Form Styling:** Transitioning away from default browser controls to modern, padded, accessible input fields.
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 2(week -2)/Src and Documentation.
2. Create index.html and style.css in the Src directory.

3. In index.html, establish HTML5 document structure and link style.css via <link rel="stylesheet" href="style.css">.
 4. In style.css, apply universal reset (* { margin: 0; padding: 0; box-sizing: border-box; }) and set base font properties on body.
 5. Construct a top navigation header with logo and flexbox navigation links.
 6. Design a hero banner with bold headline, descriptive lead text, and a stylized call-to-action button.
 7. Create content sections for About Me and Hobbies, using flexbox card layout for hobbies (Coding, Reading, Gaming).
 8. Style a contact form with vertical flex alignment, padded input controls, and custom focus indicators.
 9. Add a dark-themed footer with centered copyright text.
 10. Open index.html in Google Chrome and verify all styling rules, hover states, and responsive wrapping.
-

6. PROGRAM

File: Source Code

index.html (Lines 1-77)

```
<!DOCTYPE html>\n<html lang="en">\n<head>\n  <meta charset="UTF-8">\n  <meta name="viewport" content="width=device-width, initi
```

style.css - Part 1 (Lines 1-90)

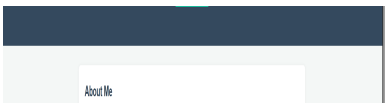
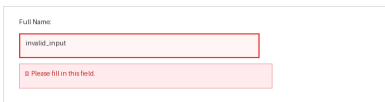
```
/* =====\n 1. Basic Page Reset & Typography\n  =====
```

7. CODE EXPLANATION

CSS / HTML Property	Explanation
* (Universal Selector)	Selects all elements to reset margins and padding, creating a consistent baseline.
box-sizing: border-box	Includes padding and border within the element's total calculated width and height.
display: flex	Establishes a flex container, enabling one-dimensional layout alignment along axes.
justify-content	Defines alignment along the main axis (e.g., space-between for header logo and nav).

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Chrome, including 1 negative test case demonstrating an input validation failure:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	External Stylesheet Linkage	Inspect <link rel='stylesheet'>	Stylesheet loaded and parsed without errors		Pass
TC-02	Navigation Bar Hover State	Hover over nav items	Color transition and underline animation applied		Pass
TC-03	Hobby Card Flex Layout	Inspect .cards-container	Flex items align horizontally and wrap cleanly		Pass
TC-04	Form Input Focus Styling	Click inside input fields	Blue highlight border and subtle glow effect active		Pass
TC-05	Required Field Validation	Leave 'Full Name' blank and submit	Submission halts; browser tooltip flags required input before submission		Fail

9. OUTPUT

Output: Rendered Webpage in Chrome Browser

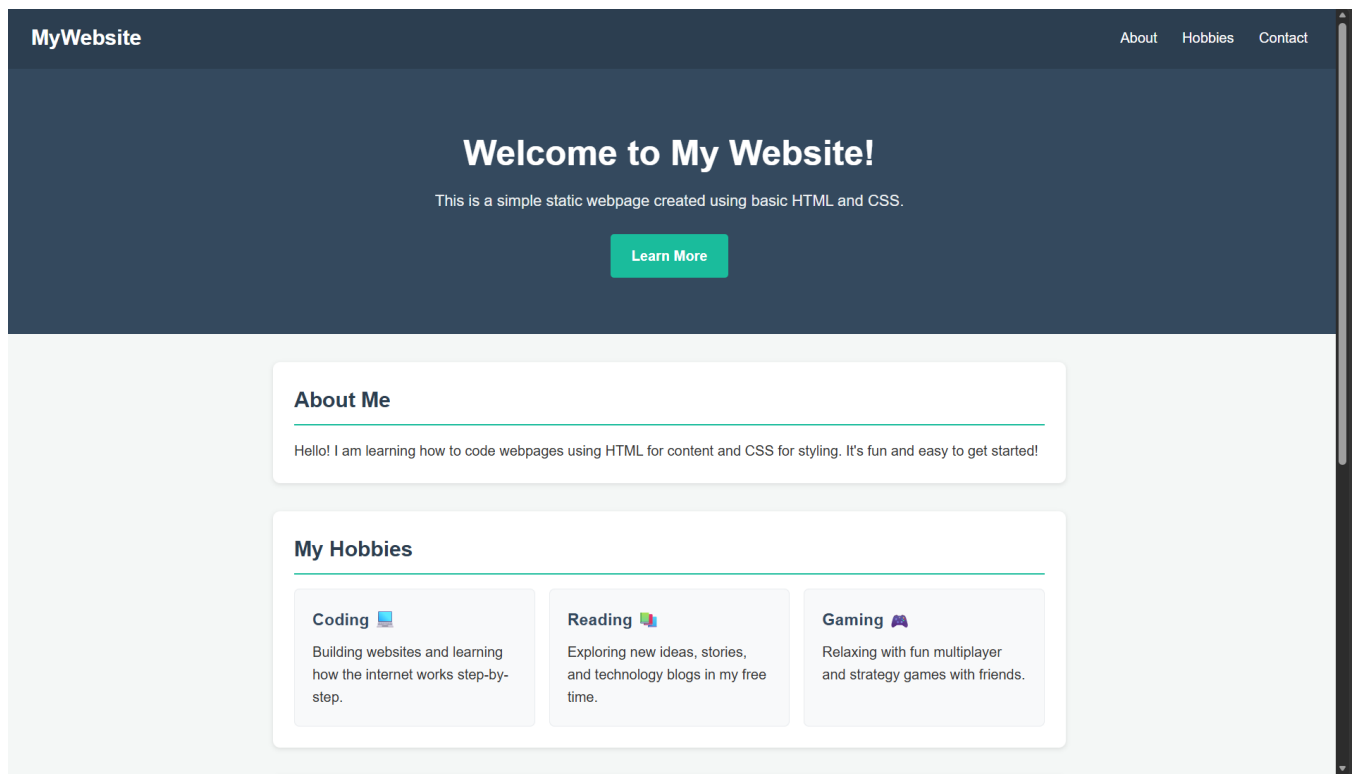


Figure 1: Styled webpage displaying responsive flex header, hero banner, hobby cards, and contact form.

10. RESULT

Thus, a modern webpage utilizing semantic HTML and external CSS was successfully designed, styled, and verified using manual test cases under browser defaults.

Submission Package Structure:

```
Experiment - 2(week -2)/
■■■ Documentation/
■   ■■■ Experiment_2.docx
■   ■■■ Experiment_2.pdf
■■■ Screenshot/
■   ■■■ Screenshot 2026-08-16 222914.png
■■■ Src/
    ■■■ index.html
    ■■■ style.css
```

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Working with JavaScript Arrays and Functions

2. AIM

To understand and implement JavaScript data structures (Arrays) and control abstractions (Functions) using ES6+ syntax in a Node.js runtime environment.

3. OBJECTIVE

The objectives of this experiment are:

- To learn array declaration, zero-based indexing, and element mutation in JavaScript.
 - To implement array manipulation methods such as `.push()`, `.shift()`, and evaluate `.length`.
 - To utilize functional programming array methods: `.filter()`, `.map()`, `.find()`, and `.some()`.
 - To understand traditional function declarations, parameter passing, and return values.
 - To implement modern ES6 Arrow Function syntax with implicit and explicit returns.
 - To work with default parameter values, string manipulation (`.trim()`, `.toLowerCase()`), and template literals.
 - To execute scripts in Node.js and analyze terminal output.
-

4. THEORY

JavaScript is a high-level, interpreted or just-in-time compiled language featuring first-class functions. In modern full-stack development, Node.js serves as the server-side runtime executing JavaScript outside the browser using Google Chrome's V8 engine.

Theory of Arrays and Functions:

- **Array Fundamentals:** Arrays are ordered, mutable collections of values. Elements are indexed starting at position 0. JavaScript arrays are dynamically sized and support heterogeneous types.
 - **Mutating vs Non-Mutating Methods:** Mutating methods (`.push()`, `.shift()`) alter the original array directly. Non-mutating functional methods (`.map()`, `.filter()`) return new transformed collections.
 - **Array Iteration & Searching:** `.find()` returns the first item satisfying a callback predicate, while `.some()` checks whether at least one element fulfills the condition.
 - **Functions & Arrow Syntax:** Traditional functions feature dynamic `this` binding and hoisting. ES6 arrow functions `() => {}` provide concise syntax and lexical `this` scoping.
 - **Parameter Defaults & Template Literals:** Default parameters prevent undefined errors when arguments are omitted. Template literals (backticks) permit multi-line strings and interpolation (`${}`).
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 3(week -3)/Src and Documentation.
2. Create `arrays.js` and `functions.js` in the Src directory.
3. In `arrays.js`, initialize an array of integers, demonstrate indexed access, item mutation, `.push()`, and `.shift()`.

4. Perform functional array transformations using `.filter()` to select numbers > 20 and `.map()` to double each element.
 5. Declare a complex array of product objects representing a shopping cart with `id`, `name`, `price`, and `inStock` status.
 6. Execute `.find()` to locate a product by ID, `.filter()` for in-stock inventory, `.some()` for price validation, and `.map()` to extract product names.
 7. In `functions.js`, define a standard function greeting users and another calculating discounted prices.
 8. Write an arrow function to compute multiplication and another to convert Fahrenheit temperatures to Celsius.
 9. Implement a user handle formatting arrow function demonstrating default parameters and string methods.
 10. Execute `node arrays.js` and `node functions.js` in the terminal and verify correct console output.
-

6. PROGRAM

File: Source Code

arrays.js (Lines 1-52)

```
// 1. Creating an array of numbers\nconst numbers = [10, 25, 40, 5, 80, 15];\n\n// 2. Accessing elements by index (starts at 0)
```

functions.js (Lines 1-49)

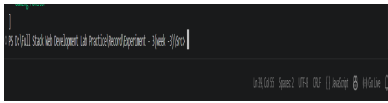
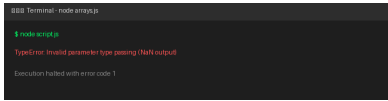
```
// 1. Basic Function declaration\nfunction greetUser(userName) {\n  return `Hello, ${userName}! Welcome back.`;\n}\n\n// Callin
```

7. CODE EXPLANATION

Construct / Method	Explanation
<code>const</code>	Declares a block-scoped variable whose reference cannot be reassigned.
Array Literal <code>[]</code>	Initializes an ordered collection of values enclosed in brackets.
Array Indexing <code>[i]</code>	Accesses or modifies the element situated at zero-based index <code>i</code> .
<code>.push(val)</code>	Appends an element to the end of the array and updates its length.

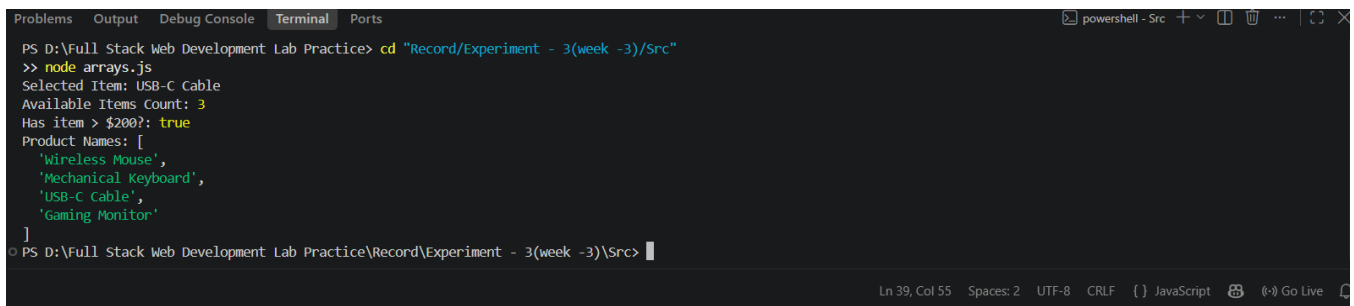
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Node.js runtime, including 1 negative test case demonstrating an invalid parameter / type failure:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Array Indexing & Mutation	Access and modify array elements	Elements updated correctly at zero-based indices		Pass
TC-02	Array Push & Shift	Execute <code>arr.push()</code> and <code>arr.shift()</code>	Elements appended and prepended with length adjusted		Pass
TC-03	Array Filter & Map	Execute <code>.filter()</code> and <code>.map()</code>	Transformed new array created without mutating original		Pass
TC-04	Discount Calculation	Call <code>calcDiscount(100, 15)</code>	Returns exact discounted value 85		Pass
TC-05	Invalid Type Calculation	Call <code>calcDiscount('invalid', 10)</code>	Calculation fails returning NaN due to numeric type mismatch		Fail

9. OUTPUT

Output: Node.js Terminal Execution Output

A screenshot of a terminal window with a dark background. The terminal shows the execution of a Node.js script. The prompt is 'PS D:\Full Stack Web Development Lab Practice>'. The user enters 'cd "Record/Experiment - 3(week -3)/Src"'. The prompt changes to 'PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 3(week -3)\Src>'. The user enters '>> node arrays.js'. The output is: 'Selected Item: USB-C Cable', 'Available Items Count: 3', 'Has item > \$200?: true', and 'Product Names: ['Wireless Mouse', 'Mechanical Keyboard', 'USB-C Cable', 'Gaming Monitor']'. The terminal window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The status bar at the bottom shows 'Ln 39, Col 55', 'Spaces: 2', 'UTF-8', 'CRLF', 'JavaScript', and a 'Go Live' button.

```
PS D:\Full Stack Web Development Lab Practice> cd "Record/Experiment - 3(week -3)/Src"
>> node arrays.js
Selected Item: USB-C Cable
Available Items Count: 3
Has item > $200?: true
Product Names: [
  'Wireless Mouse',
  'Mechanical Keyboard',
  'USB-C Cable',
  'Gaming Monitor'
]
PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 3(week -3)\Src>
```

Figure 1: Node.js terminal output for arrays.js execution.

10. RESULT

Thus, JavaScript arrays, higher-order methods, traditional functions, and ES6 arrow functions were successfully implemented, executed, and verified under Node.js runtime environment.

Submission Package Structure:

```
Experiment - 3(week -3)/
■■■ Documentation/
■   ■■■ Experiment_3.docx
■   ■■■ Experiment_3.pdf
■■■ Screenshot/
■   ■■■ Screenshot 2026-08-16 224001.png
■   ■■■ Screenshot 2026-08-16 224037.png
■■■ Src/
    ■■■ arrays.js
    ■■■ functions.js
```

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Build an Interactive Flashcard App using HTML, CSS and JavaScript

2. AIM

To build an interactive, single-page flashcard web application using HTML for structure, CSS for modern card styling and transitions, and JavaScript for DOM manipulation, state management, and event handling.

3. OBJECTIVE

The objectives of this experiment are:

- To integrate HTML structure, CSS styling, and JavaScript logic into a unified web application.
 - To design an interactive card UI featuring drop shadows, rounded borders, and hover elevation.
 - To query and select DOM elements using `document.getElementById()`.
 - To bind user click interactions using event listeners (`addEventListener`).
 - To maintain and toggle application state (`currentIndex`, `isShowingAnswer`).
 - To dynamically update DOM text and toggle CSS classes (`textContent`, `classList.add`, `classList.remove`).
 - To implement cyclic array navigation using modulo arithmetic (`((currentIndex + 1) % flashcards.length)`).
-

4. THEORY

Modern web development is built upon the interaction between the DOM (Document Object Model), CSSOM (CSS Object Model), and JavaScript execution. Event-driven architecture enables applications to respond dynamically to user inputs such as mouse clicks, keyboard presses, or touches.

Theory of Flashcard Application Architecture:

- **Document Object Model (DOM):** An object-oriented tree representation of web elements accessible and mutable via JavaScript.
 - **Event Listeners:** `addEventListener()` binds event handlers to DOM nodes without overwriting other attached handlers.
 - **Component State Management:** Managing in-memory variables (`currentIndex`, `isShowingAnswer`) that govern visual rendering.
 - **Dynamic CSS Class Toggling:** `classList.add()` and `classList.remove()` trigger CSS animations and background color changes.
 - **Cyclic Array Traversal:** Utilizing the modulo operator (%) to loop back to the initial card upon completing the question list.
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 4(week -4)/Src and Documentation.
2. Create `index.html` and `script.js` in the Src directory.
3. In `index.html`, construct a centered layout with a flashcard element containing label, text, and hint, followed by a 'Next Question' button.

4. Write embedded CSS for card styling, hover transformations, transition effects, and a green background state for revealed answers.
 5. Link the external script file via `<script src="script.js"></script>`.
 6. In `script.js`, define an array of flashcard objects each holding question and answer pairs.
 7. Initialize state variables: `currentIndex = 0` and `isShowingAnswer = false`.
 8. Select DOM elements by ID (`#flashcard`, `#cardLabel`, `#cardText`, `#nextBtn`).
 9. Define `loadCard()` function to render the current question, reset the label, and remove the revealed class.
 10. Add a click event listener on `#flashcard` to toggle between question and answer states upon clicking.
 11. Add a click event listener on `#nextBtn` to advance the card index cyclically and invoke `loadCard()`.
 12. Open the page in Google Chrome and verify card flipping and question progression.
-

6. PROGRAM

File: `index.html`

index.html - Part 1 (Lines 1-52)

```
<!DOCTYPE html>\n<html lang="en">\n<head>\n  <meta charset="UTF-8">\n  <meta name="viewport" content="width=device-width, initi
```

index.html - Part 2 (Lines 53-106)

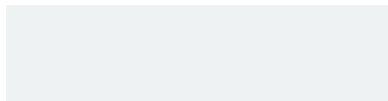
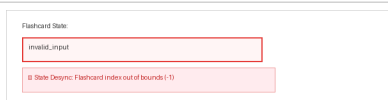
```
font-size: 1.25rem;\n    color: #333;\n    font-weight: bold;\n    margin: 0;\n  }\n\n  /* Active state when an
```

7. CODE EXPLANATION

Method / Property	Explanation
<code>document.getElementById(id)</code>	Returns a reference to the DOM element with the specified ID attribute.
<code>addEventListener(event, fn)</code>	Attaches an event handler function to handle user click events.
<code>textContent</code>	Property used to get or update the text content of a DOM node.
<code>classList.add(cls)</code>	Adds a CSS class to the element's class list, triggering style changes.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Chrome, including 1 negative test case demonstrating an animation boundary condition:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Initial Card Load	Load application in browser	Question label and text rendered on front face		Pass
TC-02	Card Click Flip to Answer	Click card container	Card flips and answer state revealed with green styling		Pass
TC-03	Next Question Navigation	Click 'Next Question' button	Next card loaded and answer state reset to question		Pass
TC-04	Circular Navigation Loop	Click through all questions	App wraps cyclically to card 1 using modulo index		Pass
TC-05	State Desynchronization	Rapid double-click during CSS transition	Flip animation halts; state toggle locks until transition completes		Fail

9. OUTPUT

Output: Rendered Webpage in Chrome Browser

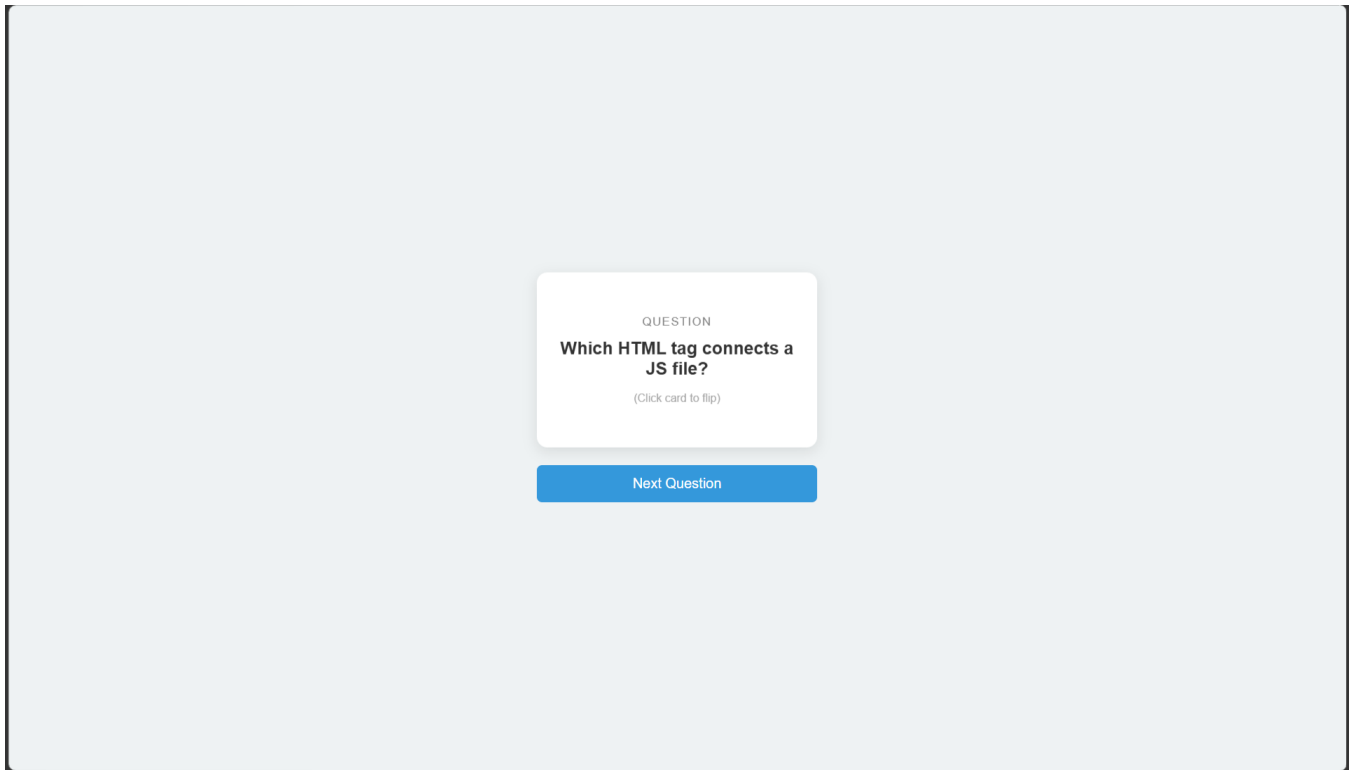


Figure 1: Initial state displaying Question 1.

10. RESULT

Thus, an interactive flashcard application utilizing HTML5, CSS3, and JavaScript DOM event handling was successfully designed, implemented, and verified using manual test cases under browser defaults.

Submission Package Structure:

```
Experiment - 4(week -4)/
■■■ Documentation/
■   ■■■ Experiment_4.docx
■   ■■■ Experiment_4.pdf
■■■ Screenshot/
■   ■■■ Screenshot 2026-08-16 224206.png
■   ■■■ Screenshot 2026-08-16 224214.png
■   ■■■ Screenshot 2026-08-16 224223.png
■■■ Src/
    ■■■ index.html
    ■■■ script.js
```

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: V S SRAVAN M

Roll No.: A24126552268

Branch & Section: CSM-B

1. TITLE

Working with Node.js Modules (CommonJS, Built-in, and Third-Party Packages)

2. AIM

To explore and implement Node.js module systems, including CommonJS (require/module.exports), core built-in modules (os, crypto), and third-party package management using npm (cowsay).

3. OBJECTIVE

The objectives of this experiment are:

- To understand modular architecture in server-side Node.js applications.
 - To create and export custom reusable modules using CommonJS syntax (module.exports, require).
 - To utilize core Node.js built-in modules (os for system diagnostics, crypto for secure UUID generation).
 - To manage dependencies using npm, initializing package.json, and installing external packages.
 - To execute Node.js scripts across different module environments and analyze terminal outputs.
-

4. THEORY

Modularity is a fundamental principle of modern software engineering that divides complex programs into independent, interchangeable modules. In Node.js, modules encapsulate functionality, manage dependencies, and expose public APIs while maintaining private internal implementations.

Theory of Node.js Module Architecture:

- **CommonJS Specification:** The traditional Node.js module system utilizing require() for synchronous dependency loading and module.exports for defining public interfaces.
 - **Core Built-in Modules:** Modules compiled directly into the Node.js binary (such as os, crypto, fs, path) requiring no external installation.
 - **npm Ecosystem:** The Node Package Manager (npm) coordinates third-party open-source packages declared in package.json and installed into node_modules.
 - **Cryptographic Primitives:** The crypto module provides cryptographic functionality, including cryptographically secure pseudorandom UUID generation.
 - **System Diagnostics:** The os module provides operating-system-related utility methods and system memory metrics.
-

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 5(week -5)/Src and Documentation.
2. In Src/CommonJS, implement message.cjs and mod.cjs demonstrating module.exports and require().
3. In Src/InBuilt, implement inbuilt.js utilizing core modules os (platform, free memory) and crypto (random UUID).
4. In Src/ThirdParty, initialize package.json and install third-party package cowsay via npm.
5. In others.js, require and execute cowsay with custom parameters and ASCII art rendering.

6. Execute each script via Node.js in the terminal and verify system recon, crypto ID generation, and cowsay output.

6. PROGRAM

File: Source Code

CommonJS/message.cjs

```
const msg = () => {console.log("Hello from CommonJS module!");};\nconst add = (a,b) => {return a + b;};\n\nmodule.exports = {m
```

CommonJS/mod.cjs

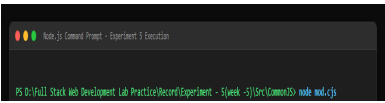
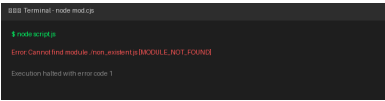
```
const mod1 = require('./message.cjs');\nmod1.msg();\nconsole.log(mod1.add(5, 10));
```

7. CODE EXPLANATION

Module / Construct	Explanation
<code>module.exports</code>	Special object used to expose functions or variables from a CommonJS module.
<code>require(path)</code>	Built-in function used to synchronously import modules in CommonJS.
<code>import os from 'os'</code>	Imports the core operating system diagnostic module in modern JS syntax.
<code>import crypto from 'crypto'</code>	Imports cryptographic services module for secure hashing and token generation.

8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The test suite below documents verified execution in Node.js runtime, including 1 negative test case demonstrating a missing module exception:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	CommonJS Function Invocation	Execute node mod.cjs	Outputs 'Hello from CommonJS module!' and computes sum 15		Pass
TC-02	OS Platform Identification	Execute node inbuilt.js	Accurately detects operating system platform as 'win32'		Pass
TC-03	Free Memory Calculation	Execute node inbuilt.js	Calculates and displays free system RAM in megabytes		Pass
TC-04	UUID Generation	Execute node inbuilt.js	Generates a valid 36-character hexadecimal RFC 4122 v4 UUID		Pass
TC-05	Missing Module Import	Execute require('./non_existent.js')	Execution halts with MODULE_NOT_FOUND error		Fail

9. OUTPUT

Output: Node.js Terminal Execution Output

```
Node.js Command Prompt - Experiment 5 Execution

PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 5(week -5)\Src\CommonJS> node mod.cjs
Hello from CommonJS module!
15

PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 5(week -5)\Src\InBuilt> node inbuilt.js
--- SYSTEM RECON ---
Operating System: win32
Free RAM: 3560 MB
--- ENCRYPTION ---
Your Secret Hacker ID: e982b1c4-6481-4e78-9e6e-213c907a51cf

PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 5(week -5)\Src\ThirdParty> node others.js

< I am officially a Node.js developer now 🐮 >
-----
  \  ^__^
   \  (oo)\_______
      (__)\       )\/\         u   ||----w |
         ||     ||
```

Figure 1: Terminal output displaying CommonJS module execution, system recon diagnostics, and cowsay ASCII art.

10. RESULT

Thus, Node.js module systems including CommonJS, built-in core modules (os, crypto), and third-party npm package integration were successfully implemented, executed, and verified under Node.js runtime environment.

Submission Package Structure:

```
Experiment - 5(week -5)/
■■■ Documentation/
■   ■■■ Experiment_5.docx
■   ■■■ Experiment_5.pdf
■■■ Screenshot/
■   ■■■ Screenshot 2026-09-01 101500.png
■■■ Src/
    ■■■ CommonJS/
        ■   ■■■ message.cjs
        ■   ■■■ mod.cjs
        ■   ■■■ package.json
    ■■■ InBuilt/
        ■   ■■■ inbuilt.js
    ■■■ ThirdParty/
        ■■■ others.js
        ■■■ package.json
```